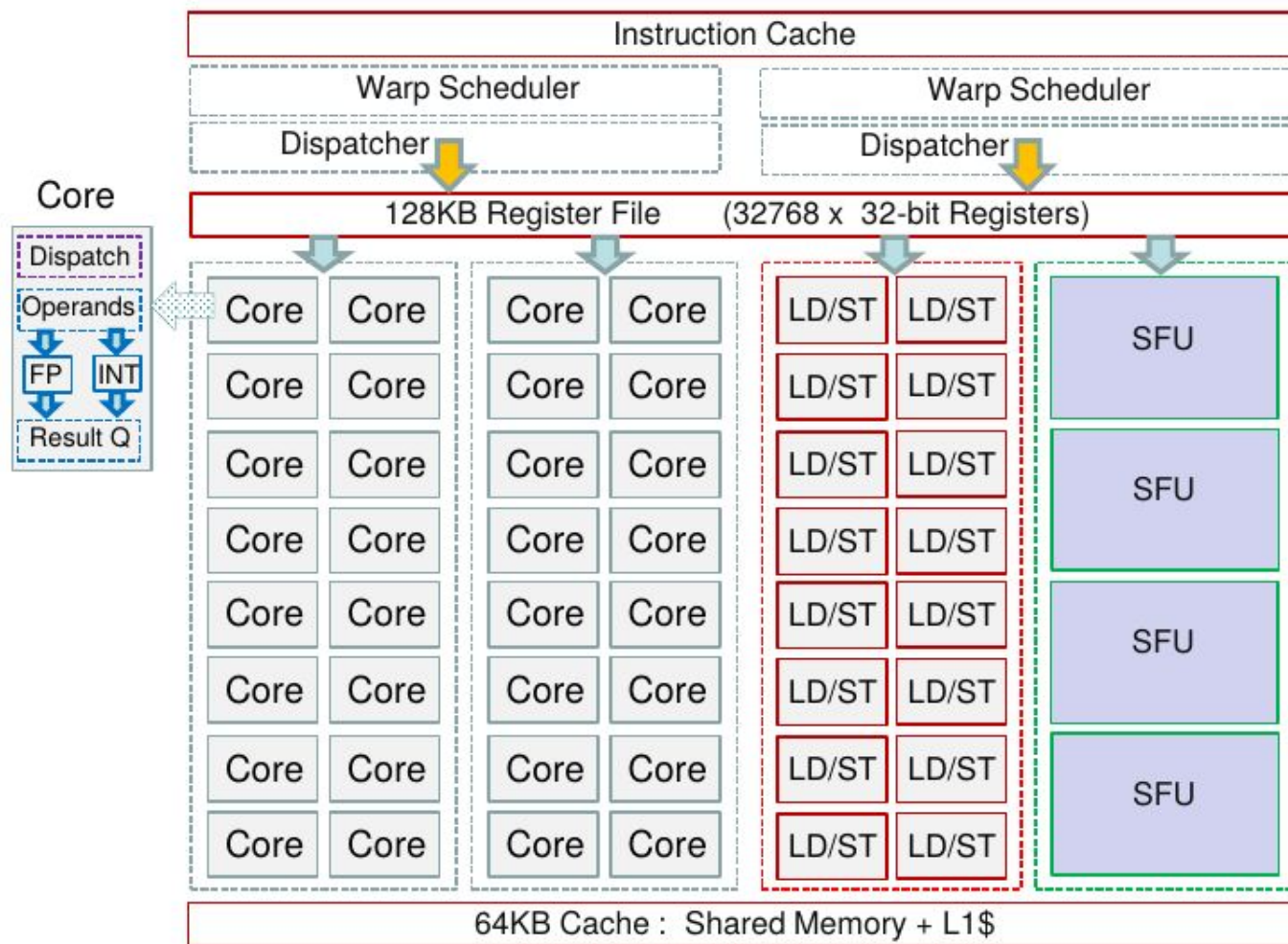
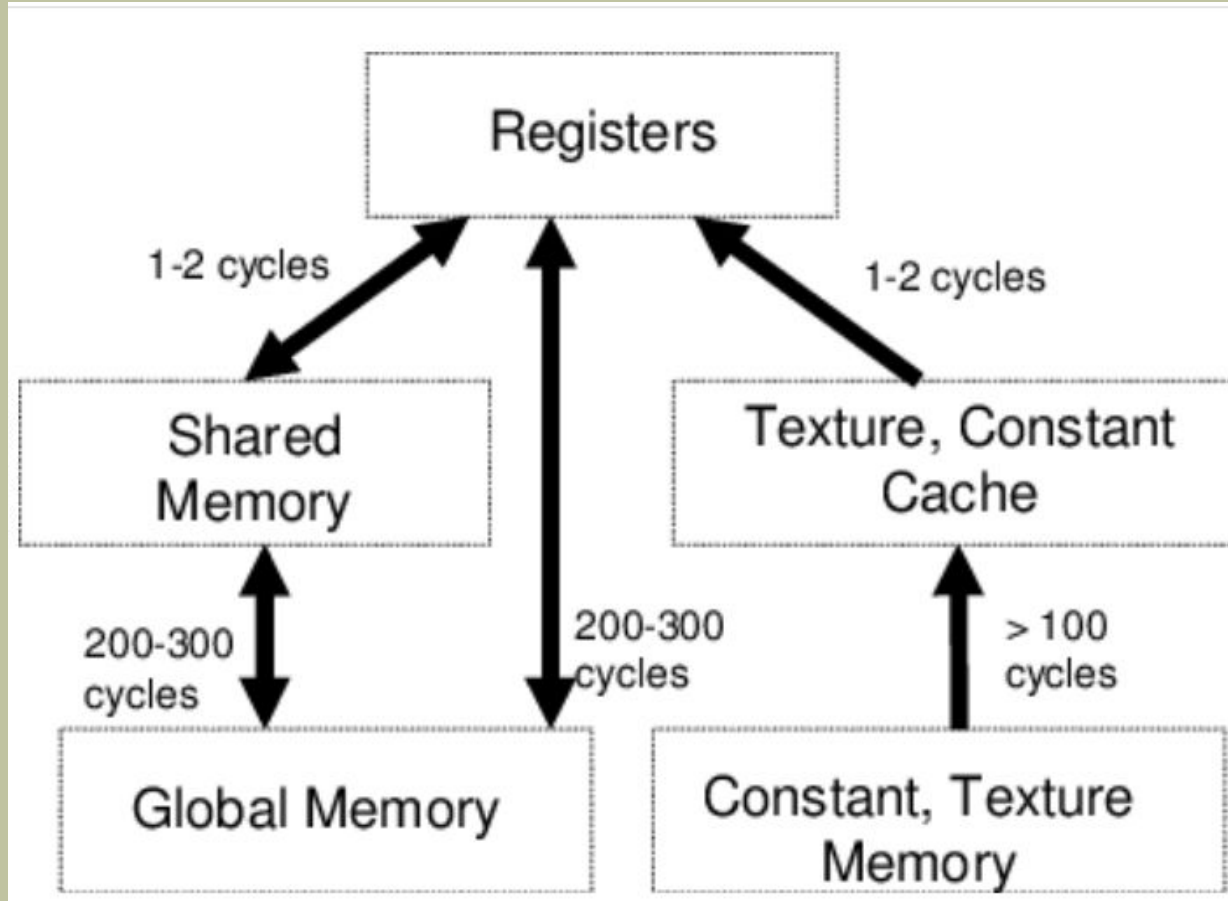


Лекция 7

- Константная память.
- Текстурная память.





```
#include <stdio.h>
```

```
#define ARRAY_SIZE 2048
```

```
#define THREADS_PER_BLOCK 64
```

test_mem.cu

```
__device__ float A[ARRAY_SIZE];
```

```
__device__ float B[ARRAY_SIZE/THREADS_PER_BLOCK];
```

```
__constant__ float C[ARRAY_SIZE/THREADS_PER_BLOCK];
```

```
__global__ void gOut();
```

```
__global__ void gInit(){
```

```
    int i= threadIdx.x+blockIdx.x*blockDim.x;
```

```
    A[i]=(float)i;
```

```
    B[blockIdx.x]=(float)blockIdx.x;
```

```
}
```

```
void hInitC(){  
    int N=ARRAY_SIZE/THREADS_PER_BLOCK;  
    float* hC=(float*)calloc(N, sizeof(float));  
    for(int n=0;n<N;n++)  
        hC[n]=(float)n;  
  
    cudaMemcpyToSymbol(C, hC, N*sizeof(float), 0,  
                        cudaMemcpyHostToDevice);  
  
    free(hC);  
}
```

```
__global__ void gTestB(){  
    int i= threadIdx.x+blockIdx.x*blockDim.x;  
    A[i]=A[i]*(B[blockIdx.x]+1);  
}
```

```
__global__ void gTestC(){  
    int i= threadIdx.x+blockIdx.x*blockDim.x;  
    A[i]=A[i]*(C[blockIdx.x]+1);  
}
```

```
int main(){
    gInit<<<ARRAY_SIZE/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>();
    cudaDeviceSynchronize();

    hInitC();

    gTestB<<<ARRAY_SIZE/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>();
    cudaDeviceSynchronize();

    gTestC<<<ARRAY_SIZE/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>();
    cudaDeviceSynchronize();

    gOut<<<ARRAY_SIZE/THREADS_PER_BLOCK, THREADS_PER_BLOCK>>>();
    cudaDeviceSynchronize();

    return 0;
}
```

```
#include <stdio.h>
```

```
#define ARRAY_SIZE 2048
```

```
#define THREADS_PER_BLOCK 64
```

test_mem2.cu

```
extern __device__ float B[ARRAY_SIZE/THREADS_PER_BLOCK];
```

```
extern __constant__ float C[ARRAY_SIZE/THREADS_PER_BLOCK];
```

```
__global__ void gOut(){
```

```
    printf("B %g\tC: %g\n", B[blockIdx.x], C[blockIdx.x]);
```

```
}
```

```
~/Lab7> nvcc -rdc=true test_mem.cu test_mem2.cu -o test_mem
```

```
7> nvprof ./test_mem > test.txt
```

```
==9251== NVPROF is profiling process 9251, command: ./test_mem
```

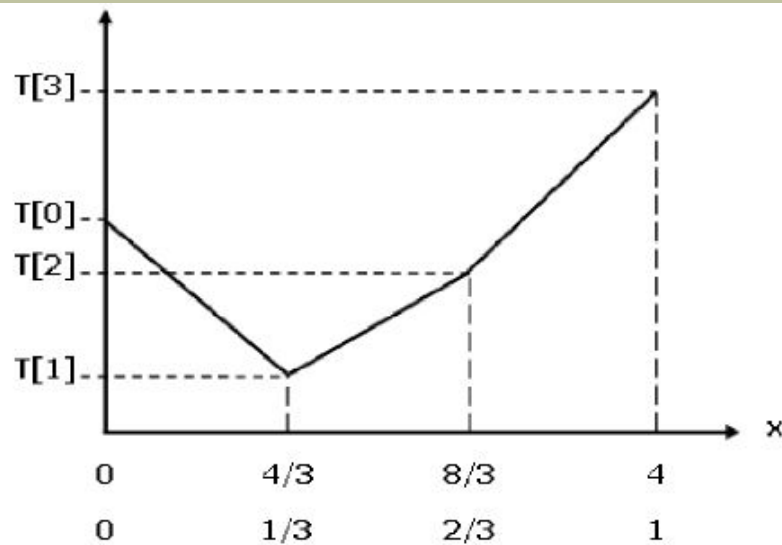
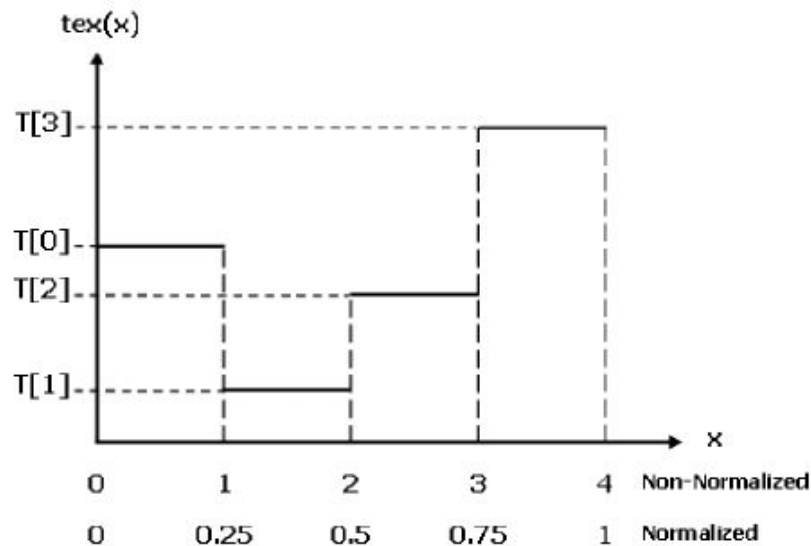
```
==9251== Profiling application: ./test_mem
```

```
==9251== Profiling result:
```

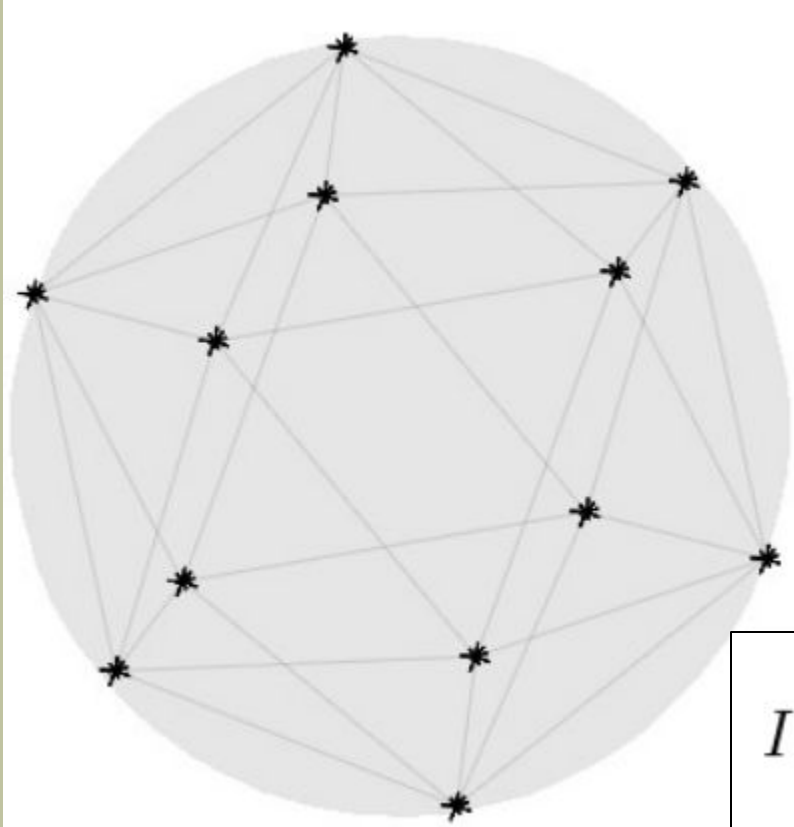
Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	99.84%	2.8895ms	1	2.8895ms	2.8895ms	2.8895ms	
gOut(void)							
	0.05%	1.5360us	1	1.5360us	1.5360us	1.5360us	gInit(void)
	0.05%	1.4080us	1	1.4080us	1.4080us	1.4080us	gTestB(void)
	0.04%	1.2160us	1	1.2160us	1.2160us	1.2160us	gTestC(void)

Текстурная память

- Текстура – специальный интерфейс доступа к глобальной памяти, обеспечивающий 1D, 2D и 3D целочисленную и **вещественную** индексацию.



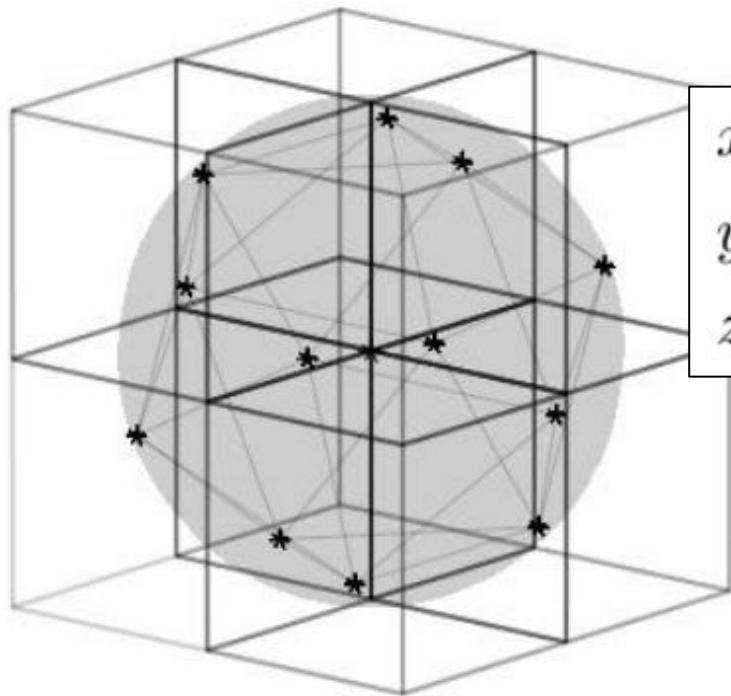
Интегрирование по сфере



Интеграл по сфере функции $g(\phi, \psi)$:

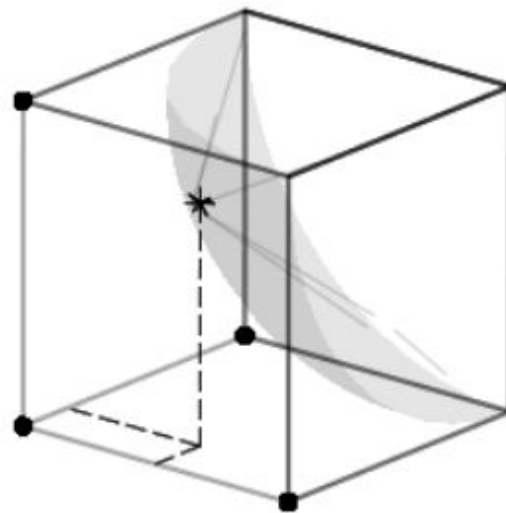
$$I = \int_{S^2} g(\phi, \psi) |\sin(\psi)| d\phi d\psi \approx \sum_i g_i (\Delta s)_i$$

Текстуры: аппаратная интерполяция (пример использования)



$$\begin{aligned}x &= \sin(\psi) \cos(\phi), \\y &= \sin(\psi) \sin(\phi), \\z &= \cos(\psi).\end{aligned}$$

Необходимость интерполяции:



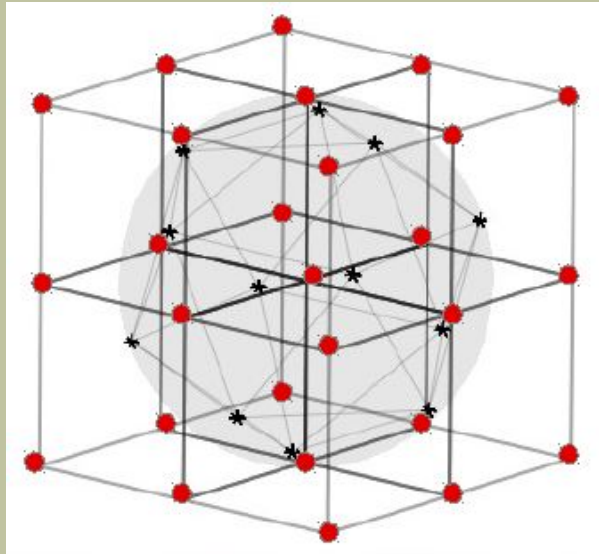
Тестовые функции, заданные в 3D пространстве

$$d_{yz} = \frac{1}{2} \sqrt{\frac{15}{\pi}} \frac{yz}{r^2}; \quad d_{z^2} = \frac{1}{4} \sqrt{\frac{5}{\pi}} \frac{-x^2 - y^2 + 2z^2}{r^2}; \quad d_{x^2 - y^2} = \frac{1}{4} \sqrt{\frac{15}{\pi}} \frac{x^2 - y^2}{r^2}.$$

Вещественные сферические функции образуют **ортонормированный** базис в гильбертовом пространстве. Первые функции в разложении по этому базису (“поперечно-скошенная”, квадрупольная и бароподобная моды).

Задание: определить эффективность аппаратной интерполяции при использовании текстурной памяти.

Шаг 1: сохранить значения тестовой функции/функций в узлах трехмерной сетки в глобальной памяти.



Шаг 2: определить значения декартовых координат узлов сферической координатной сетки на сфере радиуса RADIUS.

```
struct Vertex  
{  
    float x, y, z;  
};
```

```
__constant__ Vertex vert[VERTCOUNT];
```

```
void init_vertices(){  
    Vertex *temp_vert = (Vertex *)malloc(sizeof(Vertex) * VERTCOUNT);  
    int i = 0;
```

```
for (int iphi = 0; iphi < 2 * COEF; ++iphi)
    for (int ipsi = 0; ipsi < COEF; ++ipsi, ++i) {
        float phi = iphi * M_PI / COEF;
        float psi = ipsi * M_PI / COEF;
        temp_vert[i].x = RADIUS * sinf(psi) * cosf(phi);
        temp_vert[i].y = RADIUS * sinf(psi) * sinf(phi);
        temp_vert[i].z = RADIUS * cosf(psi);
    }
cudaMemcpyToSymbol(vert, temp_vert, sizeof(Vertex) * VERTCOUNT, 0,
                    cudaMemcpyHostToDevice);
free(temp_vert);
}
```

Шаг 3: сохранить значения тестовой функции/функций, определенные на первом шаге в текстуре.

```
cudaArray* df_Array = 0;
```

```
void load_texture(float *df_h){  
    const cudaExtent volumeSize = make_cudaExtent(FGSIZE, FGSIZE,  
                                                    FGSIZE);  
    cudaChannelFormatDesc channelDesc = cudaCreateChannelDesc<float>();  
    cudaMalloc3DArray(&df_Array, &channelDesc, volumeSize);  
    cudaMemcpy3DParms cpyParams={0};
```

```
cpyParams.srcPtr = make_cudaPitchedPtr( (void*)df_h,  
    volumeSize.width*sizeof(float),  
    volumeSize.width, volumeSize.height);  
cpyParams.dstArray = df_Array;  
  
cpyParams.extent = volumeSize;  
cpyParams.kind = cudaMemcpyHostToDevice;  
  
cudaMemcpy3D(&cpyParams);  
}
```

Шаг 4: настроить интерфейс доступа к текстуре.

```
texture<float, 3, cudaReadModeElementType> df_tex;  
  
void tune_texture(cudaChannelFormatDesc channelDesc){  
    df_tex.normalized = false;  
    df_tex.filterMode = cudaFilterModeLinear;  
    df_tex.addressMode[0] = cudaAddressModeClamp;  
    df_tex.addressMode[1] = cudaAddressModeClamp;  
    df_tex.addressMode[2] = cudaAddressModeClamp;  
  
    cudaBindTextureToArray(df_tex, df_Array, channelDesc);  
}
```

Шаг 5: реализовать вычисление интеграла, используя текстурную ссылку.

```
__global__ void kernel(float *a){  
    __shared__ float cache[THREADSPERBLOCK];  
    int tid = threadIdx.x + blockIdx.x * blockDim.x;  
    int cacheIndex = threadIdx.x;  
    float x = vert[tid].x;  
    float y = vert[tid].y;  
    float z = vert[tid].z;  
    cache[cacheIndex] = tex3D(df_tex, z, y, x);  
  
    __syncthreads();  
}
```

```
for (int s = blockDim.x / 2; s > 0; s >>= 1){  
    if (cacheIndex < s)  
        cache[cacheIndex] += cache[cacheIndex + s];  
    __syncthreads();  
}
```

*суммирование
посредством редукции*

```
if (cacheIndex == 0)  
    a[blockIdx.x] = cache[0];  
}
```

Шаг 6: реализовать функцию, выполняющую три-линейную интерполяцию и реализовать ядро, заменив текстурную ссылку вызовом этой функции.

Шаг 7: освободить ресурсы.

```
void release_texture(){  
    cudaUnbindTexture(df_tex);  
    cudaFreeArray(df_Array);  
}
```

Шаг 8: сравнить время вычисления интеграла с использованием аппаратной и программной реализации интерполяции.